

CSE231: Programming Assignment - 2

Add a new system call in xv6

Due date: Oct 3, 2026

Weightage: 10 marks

Question 1 — `getuptime()`

Problem Statement: Implement a system call `int getuptime(void)` that returns the number of timer ticks that have elapsed since the system booted.

Background: xv6 already maintains a global tick counter, incremented on every timer interrupt and protected by its own lock. This question only asks you to *expose* that existing counter to user space — you should not need to add any new state.

Hint (with links):

- 1) Check `kernel/trap.c` there is a `struct spinlock` and a `uint`, check their usage in `clockintr` in the same file, make sure to `release` the lock before returning.
- 2) Make sure to use `extern` when trying to access anything outside of `sysproc.c`, do not try to `#include` any `.c` file as you might get a multiple definition error.

Constraints: Access the tick counter under its existing lock, the same way the kernel already does internally.

Question 2 — `activecount()`

Problem Statement: Implement a system call `int activecount(void)` that returns the number of processes in the process table that are not in the `UNUSED` state.

Hint (with links):

- 1) Check `kernel/proc.c` take a look at the `void scheduler` to learn how to iterate over the process table. Make sure to acquire and release the relevant locks as well.
- 2) Check `kernel/proc.h` take a look at the definition of `struct proc` and `enum procstate`.

Constraints: Must not modify any process's state while counting, and must acquire/release each process's lock as you inspect it.

Question 3 — `lineage(int pid)`

Problem Statement: Every process in xv6, except the very first one, has a parent. Implement a system call `int lineage(int pid)` that, given a PID, prints the *full ancestry* of that process — starting from the process itself and walking up through parent, grandparent, and so on, until it reaches PID 1 (`init`). Each line should show the PID and name of one ancestor. Return the number of ancestors printed (including the process itself), or `-1` if the given PID does not correspond to an active process.

Background: This question doesn't require any new fields — the PCB already stores a pointer to a process's parent. What it asks you to design is the *traversal*: how do you follow that chain safely, one process at a time, without holding more than one process's lock at a time and without walking off the end if the chain is broken mid-way?

Hint (with links):

- 1) Same hints from Q2
- 2) Make sure to use `safestrcpy` to copy the name for printing
- 3) Make sure to not traverse further if the process is `UNUSED`.
- 4) You can assume some fixed max length of a process' name.
- 5) Use `argint` to parse the arguments of the syscall.

Constraints: Hold at most one process's lock at a time during the traversal. Handle the edge case where the given PID no longer exists by the time you look it up.

Question 4 — `getprocsz(int pid)`

Problem Statement: Implement a system call `int getprocsz(int pid)` that, given a process ID, returns the total virtual memory size in bytes allocated to that process (`p->sz`). Return `-1` if the specified PID does not exist or corresponds to a process in the `UNUSED` state.

Background: In xv6, every process maintains its virtual address space size inside its Process Control Block (`struct proc`) using the `sz` field (measured in bytes). This field grows when a process calls `sbrk()` to allocate dynamic memory. Exposing this value via a system call allows user-space utilities to monitor process memory usage without accessing kernel structures directly.

Hint (with links):

- 1) Use `argint(0, &pid)` to parse the target PID passed from user space.
- 2) Look at `kernel/proc.h` to see how `sz` is stored in `struct proc`.
- 3) Iterate through the process table (`proc[NPROC]`), acquiring and releasing `p->lock` for each entry as you check if `p->pid == pid` and `p->state != UNUSED`.
- 4) If a matching active process is found, read its `sz` field before releasing its lock and returning

Constraints: Hold at most one process lock at a time during the process table traversal. Return `-1` if the target process is not active or does not exist.

Question 5 — `familyheadcount(int)`

Problem Statement: Implement a system call `int familyheadcount(int)` that returns the number of *direct children* the calling process currently has that are still active (i.e., not yet reaped by `wait()`).

Background: No new PCB field is required — every child already stores a pointer back to its parent, so counting is a matter of scanning the process table for entries whose parent matches the caller, the same traversal style as Question 2, filtered differently.

Constraints: A child that has already exited but not yet been reaped (a zombie) should not be counted — only children still doing useful work count as "active" for this question.

Include this `test.c` file and add it to your `Makefile` as well, make sure this runs after creating all your system calls.

```
C/C++
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

int main() {
    int ticks = getuptime();
```

```

printf("System has been running for %d ticks\n", ticks);
int activeprocs = activecount();
printf("Number of active processes = %d \n", activeprocs);
int mypid = getpid();
printf("My PID is : %d\n", mypid);
int count = lineage(mypid);
printf("Count of Ancestors: %d\n", count);

int numchildren = 5;
for(int i = 0; i < numchildren; i++) {
    int pid = fork();
    if(pid < 0) {
        printf("test: fork failed\n");
        exit(1);
    }

    if(pid == 0) {
        for (volatile int i = 0; i < 10000000; i++);
        exit(0);
    }
}
for (volatile int i = 0; i < 10000; i++);

int firstcount = familyheadcount(mypid);
printf("Active children before exit: %d\n", firstcount);

for (int i = 0; i < numchildren; i++) {
    wait(0);
}

int secondcount = familyheadcount(mypid);
printf("Active children after exit: %d\n", secondcount);

int memsize = getprocsz(mypid);
printf("Memory size of process %d: %d bytes\n", mypid, memsize);

exit(0);
}

```

Expected Output:

```
None
xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ test
System has been running for 45 ticks
Number of active processes = 3
My PID is : 3
PID 3: test
PID 2: sh
PID 1: init
Count of Ancestors: 3
Active children before exit: 5
Active children after exit: 0
Memory size of process 3: 16384 bytes
$ test
System has been running for 62 ticks
Number of active processes = 3
My PID is : 9
PID 9: test
PID 2: sh
PID 1: init
Count of Ancestors: 3
Active children before exit: 5
Active children after exit: 0
Memory size of process 9: 16384 bytes
$ test
System has been running for 72 ticks
Number of active processes = 3
My PID is : 15
PID 15: test
PID 2: sh
PID 1: init
Count of Ancestors: 3
Active children before exit: 5
Active children after exit: 0
Memory size of process 15: 16384 bytes
$
```

Bonus Question — `pstree` (Process Tree Visualizer)

Problem Statement: Implement a user-space utility `pstree` (and any necessary supporting system call) that displays the active process hierarchy of the system in a visual tree format starting from `init` (PID 1).

The command must support command-line arguments to format the tree output dynamically.

Command-Line Arguments to Support:

- `pstree` (no flags): Displays the process tree showing only process names.
- `pstree -p`: Displays process PIDs alongside each process name in parentheses (e.g., `init(1)`).
- `pstree -m`: Displays process memory allocation size (`sz`) alongside each process name in brackets (e.g., `init [12288B]`).
- `pstree -p -m` (or combined `-pm` / `-mp`): Displays both PIDs and memory sizes.

Background: On Unix-like systems, `pstree` visualizes the process hierarchy as a tree, making parent-child relationships clear at a glance. In xv6, processes form a tree rooted at `init` (PID 1). Because user space cannot read kernel memory directly, you will need to design a mechanism for user space to safely retrieve process information from the kernel.

Expected Output:

None

```
$ pstree
init
|- sh
|- pstree
```

None

```
$ pstree -p
init(1)
|- sh(2)
|- pstree(3)
```

None

```
$ pstree -m
```

```
init [12288B]
  |- sh [16384B]
    |- pstree [20480B]
```

None

```
$ pstree -p -m
init(1) [12288B]
  |- sh(2) [16384B]
    |- pstree(3) [20480B]
```

Submission Guidelines

Submit a **SINGLE** zip named `<group_ID>_PA2.zip` (**any other extension is like 'rar' is NOT allowed. Only zip is allowed. If zip is not present, the assignment will be scored to 0**) containing every file you changed (preserving relative paths – `kernel/...`, `user/...`, `Makefile`). Code that fails to build with a clean `make` will not receive credit for the affected question. **Only 1 student per group should submit the assignment on the google classroom, other member should not even do blank turn-in.**

These Group IDs should only be used:

<https://docs.google.com/spreadsheets/d/1Ag0pNty9MsQLgr5qSVMYun2K8lEuACV5TnqlX9L1oyQ>

Marking scheme

Question	Weight
Q1 – <code>getuptime()</code>	15%
Q2 – <code>activecount()</code>	15%
Q3 – <code>lineage()</code>	25%

Q4 – <code>getprocsize()</code>	20%
Q5 – <code>familyheadcount()</code>	25%
BONUS question (1 mark)	1 additional mark beyond the standard 100-mark course total